

# Inżynieria danych

## Wykład 8: indeksy (cz. II)

Katarzyna Grygiel

Semestr letni 2025/2026

# Indeksy niezgrupowane i zgrupowane

W indeksach **zgrupowanych** kolejność danych w liściach odpowiada kolejności danych zapisanych na stronach w pliku. W indeksach **niezgrupowanych** kolejność danych na stronach nie ma znaczenia.

# Indeksy niezgrupowane i zgrupowane

W indeksach **zgrupowanych** kolejność danych w liściach odpowiada kolejności danych zapisanych na stronach w pliku. W indeksach **niezgrupowanych** kolejność danych na stronach nie ma znaczenia.

Stosowanie indeksów zgrupowanych jest szczególnie opłacalne w przypadku:

- operowania na zakresach danych i danych grupowanych,
- pobierania danych w określonym porządku,
- zapytań korzystających z wielu kolumn tabeli,
- dodawania nowych wierszy.

# Indeksy niezgrupowane i zgrupowane

W indeksach **zgrupowanych** kolejność danych w liściach odpowiada kolejności danych zapisanych na stronach w pliku. W indeksach **niezgrupowanych** kolejność danych na stronach nie ma znaczenia.

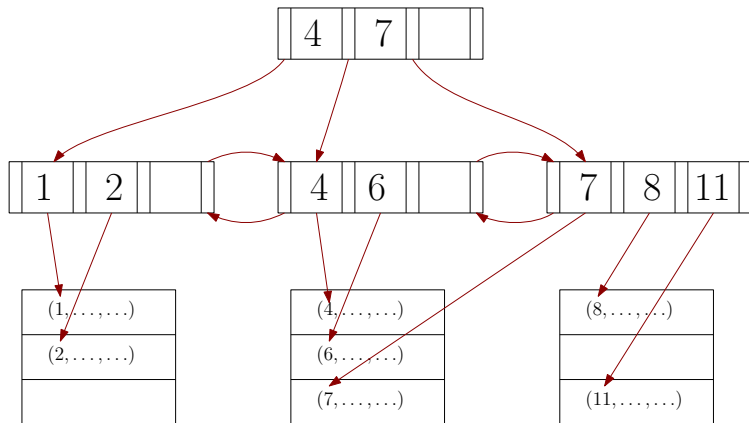
Stosowanie indeksów zgrupowanych jest szczególnie opłacalne w przypadku:

- operowania na zakresach danych i danych grupowanych,
- pobierania danych w określonym porządku,
- zapytań korzystających z wielu kolumn tabeli,
- dodawania nowych wierszy.

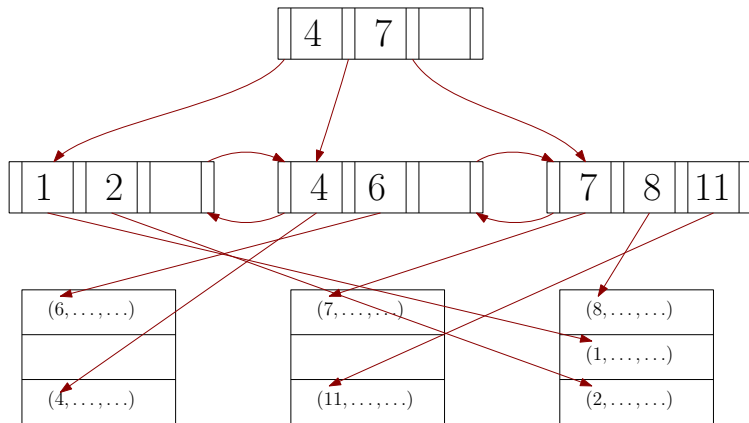
Indeks zgrupowany tworzy się na atrybutach, których wartości:

- są „krótkie”,
- rzadko się powtarzają,
- rzadko ulegają zmianie,
- są dodawane w porządku rosnącym.

## Indeks zgrupowany



## Indeks niezgrupowany



# Indeksy gęste i rzadkie

## Indeksy gęste:

- wskazują na adres rekordu danych z wartością atrybutu indeksowego równą kluczowi indeksu,
- zwykle zapewniają szybsze wyszukiwanie,
- wymagają więcej pamięci niż indeksy rzadkie,
- są trudniejsze do utrzymania przy wstawianiu i usuwaniu rekordów.

# Indeksy gęste i rzadkie

## Indeksy gęste:

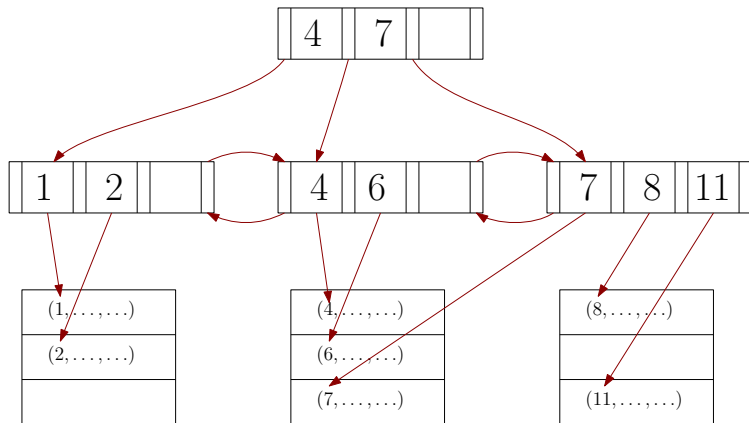
- wskazują na adres rekordu danych z wartością atrybutu indeksowego równą kluczowi indeksu,
- zwykle zapewniają szybsze wyszukiwanie,
- wymagają więcej pamięci niż indeksy rzadkie,
- są trudniejsze do utrzymania przy wstawianiu i usuwaniu rekordów.

## Indeksy rzadkie:

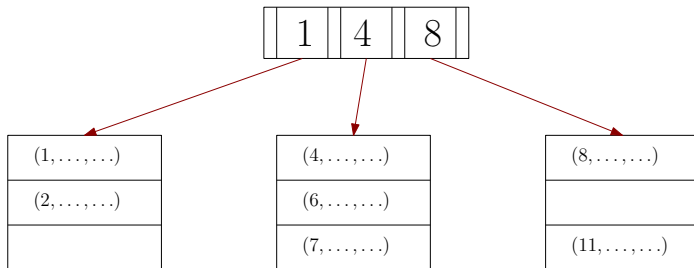
- wskazują na pierwszy blok zawierający rekord danych z wartością atrybutu indeksowego równą kluczowi indeksu,
- oszczędzają pamięć i przyspieszają wyszukiwanie rekordów.



# Indeks gęsty



# Indeks rzadki



## Optymalizacja: kompresja prefiksów

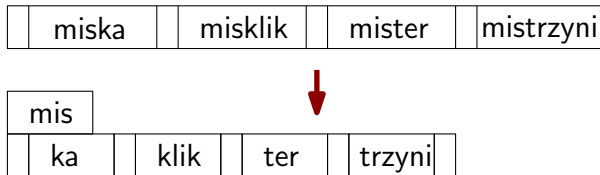
Posortowane klucze znajdujące się w tym samym węźle z dużym prawdopodobieństwem będą miały taki sam prefiks.

Zamiast przechowywać całe klucze, możemy wyekstrahować informację o maksymalnym wspólnym prefiksie i przechowywać tylko sufiksy kluczy.

## Optymalizacja: kompresja prefiksów

Posortowane klucze znajdujące się w tym samym węźle z dużym prawdopodobieństwem będą miały taki sam prefiks.

Zamiast przechowywać całe klucze, możemy wyekstrahować informację o maksymalnym wspólnym prefiksie i przechowywać tylko sufiksy kluczy.



## Optymalizacja: obcinanie sufiksów

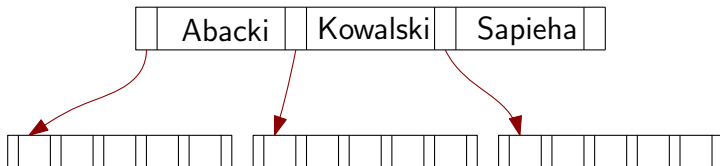
Dualnie do kompresji prefiksów, czasami możemy pozwolić sobie na obcinanie sufiksów.

W tym przypadku przechowujemy minimalny prefiks potrzebny do poprawnego odnalezienia liścia z odpowiednimi wartościami.

## Optymalizacja: obcinanie sufiksów

Dualnie do kompresji prefiksów, czasami możemy pozwolić sobie na obcinanie sufiksów.

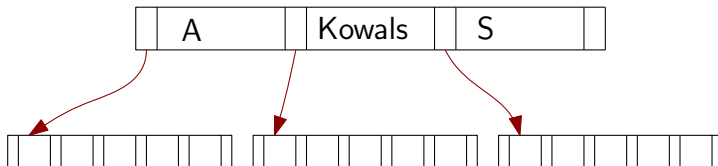
W tym przypadku przechowujemy minimalny prefiks potrzebny do poprawnego odnalezienia liścia z odpowiednimi wartościami.



## Optymalizacja: obcinanie sufiksów

Dualnie do kompresji prefiksów, czasami możemy pozwolić sobie na obcinanie sufiksów.

W tym przypadku przechowujemy minimalny prefiks potrzebny do poprawnego odnalezienia liścia z odpowiednimi wartościami.



## Optymalizacja: ładowanie zbiorcze (bulk loading)

Najszybszą metodą zbudowania B+-drzewa jest posortowanie kluczy i dopiero potem zbudowanie indeksu od dołu.



## Optymalizacja: ładowanie zbiorcze (bulk loading)

Najszybszą metodą zbudowania B+-drzewa jest posortowanie kluczy i dopiero potem zbudowanie indeksu od dołu.

Zaczynamy z kluczami

8, 2, 7, 6, 1, 11, 4,

## Optymalizacja: ładowanie zbiorcze (bulk loading)

Najszybszą metodą zbudowania B+-drzewa jest posortowanie kluczy i dopiero potem zbudowanie indeksu od dołu.

Zaczynamy z kluczami

8, 2, 7, 6, 1, 11, 4,

następnie je sortujemy:

1, 2, 4, 6, 7, 8, 11,

## Optymalizacja: ładowanie zbiorcze (bulk loading)

Najszybszą metodą zbudowania B+-drzewa jest posortowanie kluczy i dopiero potem zbudowanie indeksu od dołu.

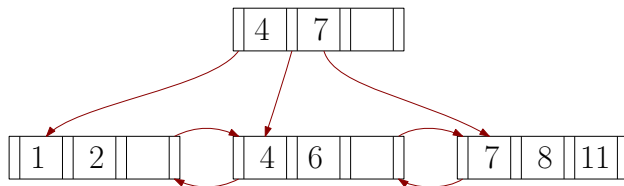
Zaczynamy z kluczami

8, 2, 7, 6, 1, 11, 4,

następnie je sortujemy:

1, 2, 4, 6, 7, 8, 11,

i dopiero teraz tworzymy indeks:



# Indeksy i sortowanie krotek

Indeksy mogą być pomocne nie tylko przy wyszukiwaniu krotek, ale również przy zwracaniu ich w żądanej kolejności (o ile oczywiście kryterium sortowania odpowiada atrybutowi, na którym założony jest indeks).

# Indeksy i sortowanie krotek

Indeksy mogą być pomocne nie tylko przy wyszukiwaniu krotek, ale również przy zwracaniu ich w żądanej kolejności (o ile oczywiście kryterium sortowania odpowiada atrybutowi, na którym założony jest indeks).

Jeśli jednak zapytanie wymaga przeskanowania dużej części tabeli, to może się okazać, że sortowanie bez wykorzystania indeksu jest szybsze ze względu na mniejszą liczbę odczytów z dysku.

W związku z tym indeksy są przydatne przy sortowaniu, jeśli wynikowy zbiór krotek jest stosunkowo nieduży.

# Indeksy i sortowanie krotek

Indeksy mogą być pomocne nie tylko przy wyszukiwaniu krotek, ale również przy zwracaniu ich w żądanej kolejności (o ile oczywiście kryterium sortowania odpowiada atrybutowi, na którym założony jest indeks).

Jeśli jednak zapytanie wymaga przeskanowania dużej części tabeli, to może się okazać, że sortowanie bez wykorzystania indeksu jest szybsze ze względu na mniejszą liczbę odczytów z dysku.

W związku z tym indeksy są przydatne przy sortowaniu, jeśli wynikowy zbiór krotek jest stosunkowo nieduży.

Indeksy mogą być tworzone według żądanego porządku dla wartości:

```
CREATE INDEX idx_studenci_imie_aktywny  
ON studenci(aktywny DESC, imie ASC);
```

## Łączenie wielu indeksów

Jeśli dla danej tabeli istnieje wiele indeksów i zapytanie dotyczy atrybutów, względem których te indeksy są utworzone, to system bazodanowy tworzy odpowiednie wektory bitowe (bitmapy) dla każdej krotki, a następnie przeprowadza na nich operacje logiczne (AND lub OR).

Dzięki temu odczyty z dysku dotyczą tylko interesujących nas krotek.

## Łączenie wielu indeksów

Jeśli dla danej tabeli istnieje wiele indeksów i zapytanie dotyczy atrybutów, względem których te indeksy są utworzone, to system bazodanowy tworzy odpowiednie wektory bitowe (bitmapy) dla każdej krotki, a następnie przeprowadza na nich operacje logiczne (AND lub OR).

Dzięki temu odczyty z dysku dotyczą tylko interesujących nas krotek.

Ponieważ jednak krotki są odczytywane z dysku na podstawie kolejności wynikającej z bitmap, porządek wyznaczony przez indeksy już nie obowiązuje – sortowanie odbywa się niezależnie od indeksów.

W związku z tym, optymalizując wykonanie zapytania, system czasami opiera się tylko na jednym z indeksów, rezygnując z wykorzystania pozostałych.



# Indeksy na wyrażeniach

Indeks może być założony nie tylko na wartościach atrybutu lub atrybutów, ale również na wartościach funkcji obliczanych na odpowiedniej kolumnie lub kolumnach.

Jest to przydatne szczególnie wtedy, kiedy zapytania dotyczą wyników pewnych obliczeń, a nie podstawowych wartości.

# Indeksy na wyrażeniach

Indeks może być założony nie tylko na wartościach atrybutu lub atrybutów, ale również na wartościach funkcji obliczanych na odpowiedniej kolumnie lub kolumnach.

Jest to przydatne szczególnie wtedy, kiedy zapytania dotyczą wyników pewnych obliczeń, a nie podstawowych wartości.

Poniższe indeksy mogą być pomocne w wyszukiwaniu odpowiednich osób na podstawie nazwiska lub w oparciu o konkatencję imienia i nazwiska:

```
CREATE INDEX idx_studenci_nazwisko
    ON studenci (LOWER(nazwisko));
CREATE INDEX idx_studenci_imie_nazwisko
    ON studenci ((imie || ' ' || nazwisko));
```

## Indeksy częściowe

Czasami niektóre dane (krotki) są dla nas bardziej interesujące niż inne. Rozważmy relację z danymi o studentach:

| <i>studenci</i> |       |             |         |
|-----------------|-------|-------------|---------|
| id              | imię  | kierunek    | aktywny |
| 1               | Zofia | matematyka  | TAK     |
| 2               | Piotr | chemia      | TAK     |
| 3               | Ewa   | matematyka  | NIE     |
| 4               | Iwo   | fizyka      | TAK     |
| 5               | Adam  | informatyka | NIE     |
| 6               | Jan   | chemia      | NIE     |
| 7               | Egon  | matematyka  | NIE     |
| 8               | Iga   | chemia      | TAK     |
| 9               | Anna  | informatyka | NIE     |

# Indeksy częściowe

Jeśli najczęściej interesują nas dane o aktywnych studentach, to zamiast tworzyć następujący indeks:

```
CREATE INDEX idx_studenci  
    ON studenci(imię);
```

warto utworzyć indeks tylko dla krotek, które odpowiadają aktywnym studentom:

```
CREATE INDEX idx_studenci_aktywny  
    ON studenci(imię)  
    WHERE aktywny = 'TAK';
```

W ten sposób utworzymy **indeks częściowy**, który zawiera tylko część danych z tabeli.

## Co za dużo...

Jeśli mamy kilka wartości atrybutu, względem których chcielibyśmy filtrować krotki do założenia indeksu częściowego, to nie jest dobrym pomysłem tworzenie wielu częściowych indeksów w następujący sposób:

```
CREATE INDEX ON tab(kryterium) WHERE atrybut = 1;  
CREATE INDEX ON tab(kryterium) WHERE atrybut = 2;  
CREATE INDEX ON tab(kryterium) WHERE atrybut = 3;  
...  
CREATE INDEX ON tab(kryterium) WHERE atrybut = N;
```

W takiej sytuacji lepiej utworzyć jeden indeks ze względu na dwa lub więcej atrybutów:

```
CREATE INDEX ON tab(atribut, kryterium);
```

## Indeksy pokrywające

Jeśli zapytanie dotyczy tylko atrybutów, na których utworzony jest indeks, to (w pewnym uproszczeniu) nie trzeba odczytywać ich wartości ze stron zawierających dane tabeli – wystarczy odczytać je z indeksu, o ile oczywiście jest to indeks, który przechowuje wartości atrybutów.

## Indeksy pokrywające

Jeśli zapytanie dotyczy tylko atrybutów, na których utworzony jest indeks, to (w pewnym uproszczeniu) nie trzeba odczytywać ich wartości ze stron zawierających dane tabeli – wystarczy odczytać je z indeksu, o ile oczywiście jest to indeks, który przechowuje wartości atrybutów.

W sytuacji, gdy zapytanie dotyczy również atrybutów niewidocznych dla indeksu, konieczne jest odczytanie krotek z miejsc na dysku, gdzie przechowywana jest tabela.

# Indeksy pokrywające

Jeśli zapytanie dotyczy tylko atrybutów, na których utworzony jest indeks, to (w pewnym uproszczeniu) nie trzeba odczytywać ich wartości ze stron zawierających dane tabeli – wystarczy odczytać je z indeksu, o ile oczywiście jest to indeks, który przechowuje wartości atrybutów.

W sytuacji, gdy zapytanie dotyczy również atrybutów niewidocznych dla indeksu, konieczne jest odczytanie krotek z miejsc na dysku, gdzie przechowywana jest tabela.

Koszt zwiększenia indeksu możemy w nim przechowywać wartości niektórych atrybutów, względem których nie jest tworzony indeks. Umożliwia to **indeksy pokrywające**.

```
CREATE INDEX idx_studenci_imie_cov_id  
ON studenci(imie) INCLUDE (id);
```



# Indeksy haszowe

**Indeksy haszowe** dobrze sprawdzają się przy zapytaniach równościowych, natomiast nie przydają się w przypadku zapytań zakresowych.

Podczas gdy wraz ze wzrostem danych rozmiar B+-drzew rośnie liniowo, rozmiar indeksów haszowych rośnie skokowo. Skoki następują w momencie, gdy przydzielana jest dodatkowa pamięć dla większej liczby kubełków.

Indeksy haszowe mają mniejszy rozmiar niż B+-drzewa. Ponadto ich rozmiar nie jest zależny od rozmiaru haszowanych wartości (jak ma to miejsce w indeksach drzewiastych).

# Indeksy haszowe

**Indeksy haszowe** dobrze sprawdzają się przy zapytaniach równościowych, natomiast nie przydają się w przypadku zapytań zakresowych.

Podczas gdy wraz ze wzrostem danych rozmiar B+-drzew rośnie liniowo, rozmiar indeksów haszowych rośnie skokowo. Skoki następują w momencie, gdy przydzielana jest dodatkowa pamięć dla większej liczby kubełków.

Indeksy haszowe mają mniejszy rozmiar niż B+-drzewa. Ponadto ich rozmiar nie jest zależny od rozmiaru haszowanych wartości (jak ma to miejsce w indeksach drzewiastych).

PostgreSQL:

```
CREATE INDEX idx_tab_atr ON tab USING HASH (atrybut);
```

# Indeksy bitmapowe

**Indeks bitmapowy** koduje w tablicy bitów (mapach bitowych) wartości krotek z zadanej kolumny (lub kolumn).

# Indeksy bitmapowe

**Indeks bitmapowy** koduje w tablicy bitów (mapach bitowych) wartości krotek z zadanej kolumny (lub kolumn).

Ponieważ wraz ze wzrostem liczby możliwych wartości rośnie rozmiar indeksu bitmapowego, stosuje się go najczęściej dla atrybutów o niewielkiej dziedzinie.

# Indeksy bitmapowe

**Indeks bitmapowy** koduje w tablicy bitów (mapach bitowych) wartości krotek z zadanej kolumny (lub kolumn).

Ponieważ wraz ze wzrostem liczby możliwych wartości rośnie rozmiar indeksu bitmapowego, stosuje się go najczęściej dla atrybutów o niewielkiej dziedzinie.

Dzięki zastosowaniu indeksów bitmapowych wykonanie niektórych zapytań sprowadza się do przeprowadzenia operacji logicznych na tablicach bitów.

Wydajność takich indeksów jest bardzo duża, jednak nie sprawdzają się tak dobrze jak np. B+-drzewa w sytuacji, gdy dane w bazie są często modyfikowane. Najczęściej stosuje się je w przypadku baz o przeznaczeniu analitycznym. Indeksy bitmapowe przechowywane są w układzie kolumnowym.

## Przykład indeksu bitmapowego

| <i>studenci</i> |       |       |
|-----------------|-------|-------|
| id              | imię  | grupa |
| 1               | Zofia | 1a    |
| 2               | Piotr | 2b    |
| 3               | Ewa   | 2a    |
| 4               | Iwo   | 1a    |
| 5               | Adam  | 1b    |
| 6               | Jan   | 2a    |
| 7               | Egon  | 2b    |
| 8               | Iga   | 2b    |
| 9               | Anna  | 1b    |

## Przykład indeksu bitmapowego

| <i>studenci</i> |       |       |
|-----------------|-------|-------|
| id              | imię  | grupa |
| 1               | Zofia | 1a    |
| 2               | Piotr | 2b    |
| 3               | Ewa   | 2a    |
| 4               | Iwo   | 1a    |
| 5               | Adam  | 1b    |
| 6               | Jan   | 2a    |
| 7               | Egon  | 2b    |
| 8               | Iga   | 2b    |
| 9               | Anna  | 1b    |

indeks bitmapowy dla grupy

| 1a | 1b | 2a | 2b |
|----|----|----|----|
| 1  | 0  | 0  | 0  |
| 0  | 0  | 0  | 1  |
| 0  | 0  | 1  | 0  |
| 1  | 0  | 0  | 0  |
| 0  | 1  | 0  | 0  |
| 0  | 0  | 1  | 0  |
| 0  | 0  | 0  | 1  |
| 0  | 0  | 0  | 1  |
| 0  | 1  | 0  | 0  |

## Przykład indeksu bitmapowego

| <i>studenci</i> |       |       |
|-----------------|-------|-------|
| id              | imię  | grupa |
| 1               | Zofia | 1a    |
| 2               | Piotr | 2b    |
| 3               | Ewa   | 2a    |
| 4               | Iwo   | 1a    |
| 5               | Adam  | 1b    |
| 6               | Jan   | 2a    |
| 7               | Egon  | 2b    |
| 8               | Iga   | 2b    |
| 9               | Anna  | 1b    |

indeks bitmapowy dla grupy

| 1a | 1b | 2a | 2b |
|----|----|----|----|
| 1  | 0  | 0  | 0  |
| 0  | 0  | 0  | 1  |
| 0  | 0  | 1  | 0  |
| 1  | 0  | 0  | 0  |
| 0  | 1  | 0  | 0  |
| 0  | 0  | 1  | 0  |
| 0  | 0  | 0  | 1  |
| 0  | 0  | 0  | 1  |
| 0  | 1  | 0  | 0  |

```
SELECT imię FROM studenci
      WHERE grupa = '1a' OR grupa = '1b';
```



# Zdekomponowany indeks bitmapowy

| <i>studenci</i> |       |        |
|-----------------|-------|--------|
| id              | imię  | miasto |
| 1               | Zofia | 3      |
| 2               | Piotr | 25     |
| 3               | Ewa   | 38     |
| 4               | Iwo   | 38     |
| 5               | Adam  | 25     |
| 6               | Jan   | 45     |
| 7               | Egon  | 3      |
| 8               | Iga   | 45     |
| 9               | Anna  | 25     |

# Zdekomponowany indeks bitmapowy

*studenci*

| id | imię  | miasto |
|----|-------|--------|
| 1  | Zofia | 3      |
| 2  | Piotr | 25     |
| 3  | Ewa   | 38     |
| 4  | Iwo   | 38     |
| 5  | Adam  | 25     |
| 6  | Jan   | 45     |
| 7  | Egon  | 3      |
| 8  | Iga   | 45     |
| 9  | Anna  | 25     |

zdekomponowany indeks dla miasta

| 0 | 1 | 2 | 3 | 4 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |

# Indeksy bitmapowe a zapytania zakresowe

*studenci*

| id | imię  | rocznik |
|----|-------|---------|
| 1  | Zofia | 2001    |
| 2  | Piotr | 1999    |
| 3  | Ewa   | 2000    |
| 4  | Iwo   | 2002    |
| 5  | Adam  | 2000    |
| 6  | Jan   | 1999    |
| 7  | Egon  | 2003    |
| 8  | Iga   | 1998    |
| 9  | Anna  | 2001    |

# Indeksy bitmapowe a zapytania zakresowe

*studenci*

| id | imię  | rocznik |
|----|-------|---------|
| 1  | Zofia | 2001    |
| 2  | Piotr | 1999    |
| 3  | Ewa   | 2000    |
| 4  | Iwo   | 2002    |
| 5  | Adam  | 2000    |
| 6  | Jan   | 1999    |
| 7  | Egon  | 2003    |
| 8  | Iga   | 1998    |
| 9  | Anna  | 2001    |

indeks kodujący zakres dla rocznika

| 1998 | 1999 | 2000 | 2001 | 2002 | 2003 |
|------|------|------|------|------|------|
| 0    | 0    | 0    | 1    | 1    | 1    |
| 0    | 1    | 1    | 1    | 1    | 1    |
| 0    | 0    | 1    | 1    | 1    | 1    |
| 0    | 0    | 0    | 0    | 1    | 1    |
| 0    | 0    | 1    | 1    | 1    | 1    |
| 0    | 1    | 1    | 1    | 1    | 1    |
| 0    | 0    | 0    | 0    | 0    | 1    |
| 1    | 1    | 1    | 1    | 1    | 1    |
| 0    | 0    | 0    | 1    | 1    | 1    |

## Indeksy bitmapowe a zapytania zakresowe

| <i>studenci</i> |       |         |
|-----------------|-------|---------|
| id              | imię  | rocznik |
| 1               | Zofia | 2001    |
| 2               | Piotr | 1999    |
| 3               | Ewa   | 2000    |
| 4               | Iwo   | 2002    |
| 5               | Adam  | 2000    |
| 6               | Jan   | 1999    |
| 7               | Egon  | 2003    |
| 8               | Iga   | 1998    |
| 9               | Anna  | 2001    |

indeks kodujący zakres dla rocznika

| 1998 | 1999 | 2000 | 2001 | 2002 | 2003 |
|------|------|------|------|------|------|
| 0    | 0    | 0    | 1    | 1    | 1    |
| 0    | 1    | 1    | 1    | 1    | 1    |
| 0    | 0    | 1    | 1    | 1    | 1    |
| 0    | 0    | 0    | 0    | 1    | 1    |
| 0    | 0    | 1    | 1    | 1    | 1    |
| 0    | 1    | 1    | 1    | 1    | 1    |
| 0    | 0    | 0    | 0    | 0    | 1    |
| 1    | 1    | 1    | 1    | 1    | 1    |
| 0    | 0    | 0    | 1    | 1    | 1    |

```
SELECT imię FROM studenci WHERE rocznik > 2000;
```

```
SELECT imię FROM studenci WHERE rocznik BETWEEN 1999 AND 2001;
```

# Filtry Blooma

**Filtr Blooma** dla zbioru  $S$  to wektor  $m$  bitów, z których początkowo wszystkie mają wartość zero. Ustalamy  $k$  różnych funkcji haszujących,  $h_1, \dots, h_k$ , o zbiorze wartości  $\{1, \dots, m\}$ . Dla każdego elementu  $s \in S$  obliczamy wartości  $h_1(s), \dots, h_k(s)$  i wszystkie bity na takich pozycjach w filtrze Blooma zapalamy na 1.

# Filtry Blooma

**Filtr Blooma** dla zbioru  $S$  to wektor  $m$  bitów, z których początkowo wszystkie mają wartość zero. Ustalamy  $k$  różnych funkcji haszujących,  $h_1, \dots, h_k$ , o zbiorze wartości  $\{1, \dots, m\}$ . Dla każdego elementu  $s \in S$  obliczamy wartości  $h_1(s), \dots, h_k(s)$  i wszystkie bity na takich pozycjach w filtrze Blooma zapalamy na 1.

Chcąc sprawdzić, czy element  $x$  należy do zbioru, obliczamy  $h_1(x), \dots, h_k(x)$  i sprawdzamy, czy wszystkie bity na tych pozycjach mają wartość 1.

Filtr Blooma pozwala nam zatem w efektywny sposób sprawdzić, czy poszukiwana wartość znajduje się w zadanym zbiorze.

# Filtry Blooma

**Filtr Blooma** dla zbioru  $S$  to wektor  $m$  bitów, z których początkowo wszystkie mają wartość zero. Ustalamy  $k$  różnych funkcji haszujących,  $h_1, \dots, h_k$ , o zbiorze wartości  $\{1, \dots, m\}$ . Dla każdego elementu  $s \in S$  obliczamy wartości  $h_1(s), \dots, h_k(s)$  i wszystkie bity na takich pozycjach w filtrze Blooma zapalamy na 1.

Chcąc sprawdzić, czy element  $x$  należy do zbioru, obliczamy  $h_1(x), \dots, h_k(x)$  i sprawdzamy, czy wszystkie bity na tych pozycjach mają wartość 1.

Filtr Blooma pozwala nam zatem w efektywny sposób sprawdzić, czy poszukiwana wartość znajduje się w zadanym zbiorze.

Metoda ta jest probabilistyczna, a tym samym obarczona ryzykiem błędu.

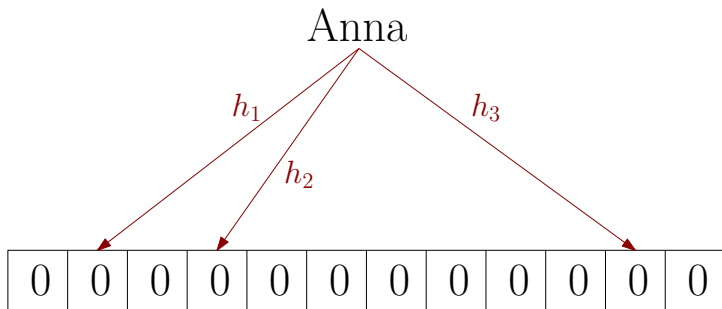


## Przykład filtru Blooma

Anna

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|

## Przykład filtru Blooma



## Przykład filtru Blooma

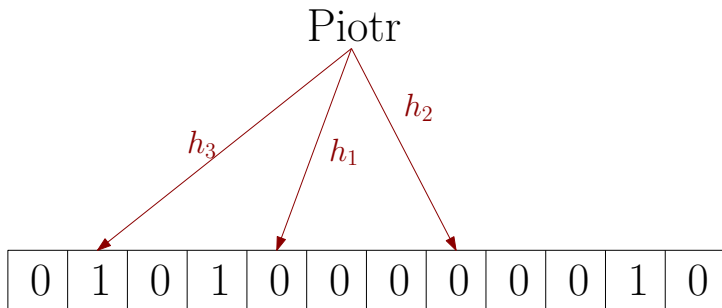
|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|

## Przykład filtru Blooma

Piotr

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|

## Przykład filtru Blooma



## Przykład filtru Blooma

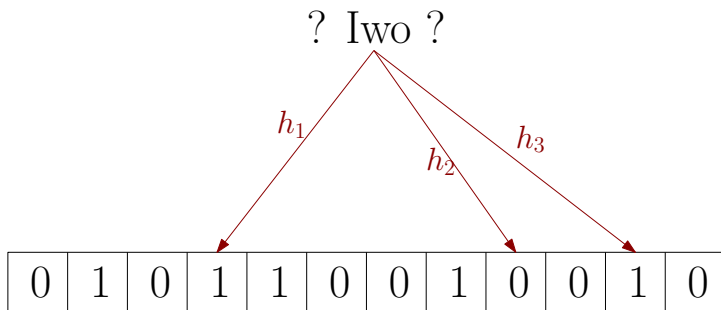
|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|

## Przykład filtru Blooma

? Iwo ?

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|

## Przykład filtru Blooma





## Przykład filtru Blooma

~~? Iwo ?~~

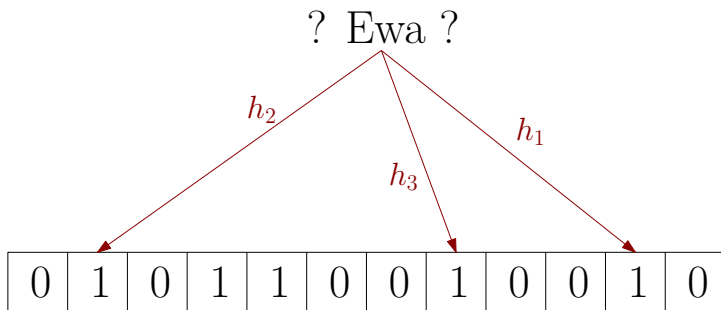
|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|

## Przykład filtru Blooma

? Ewa ?

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|

## Przykład filtru Blooma



## Przykład filtru Blooma

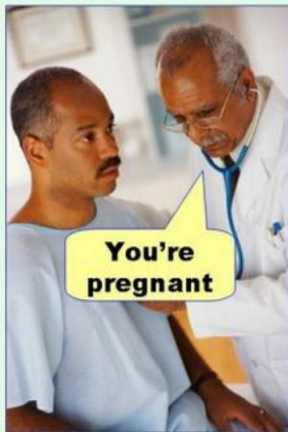
? Ewa ?

**ups...**

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|

## Błędy I i II rodzaju

**Type I error**  
(false positive)



**Type II error**  
(false negative)



## Błędy i filtry Blooma

Korzystanie z filtrów Blooma naraża nas tylko na błędy I rodzaju, czyli wyniki fałszywie dodatnie.

Oznacza to, że sprawdzając, czy zadana wartość należy do zbioru, możemy otrzymać odpowiedź „tak” nawet wtedy, gdy nie jest to prawdą. Nigdy natomiast nie uzyskamy odpowiedzi „nie”, jeśli dana wartość w rzeczywistości do zbioru należy.

## Błędy i filtry Blooma

Korzystanie z filtrów Blooma naraża nas tylko na błędy I rodzaju, czyli wyniki fałszywie dodatnie.

Oznacza to, że sprawdzając, czy zadana wartość należy do zbioru, możemy otrzymać odpowiedź „tak” nawet wtedy, gdy nie jest to prawdą. Nigdy natomiast nie uzyskamy odpowiedzi „nie”, jeśli dana wartość w rzeczywistości do zbioru należy.

Rozważmy filtr Blooma o rozmiarze  $m$  wykorzystujący  $k$  funkcji haszujących dla zbioru  $n$ -elementowego. Wówczas prawdopodobieństwo popełnienia błędu (przy założeniu niezależności odpowiednich zmiennych losowych) wynosi

## Błędy i filtry Blooma

Korzystanie z filtrów Blooma naraża nas tylko na błędy I rodzaju, czyli wyniki fałszywie dodatnie.

Oznacza to, że sprawdzając, czy zadana wartość należy do zbioru, możemy otrzymać odpowiedź „tak” nawet wtedy, gdy nie jest to prawdą. Nigdy natomiast nie uzyskamy odpowiedzi „nie”, jeśli dana wartość w rzeczywistości do zbioru należy.

Rozważmy filtr Blooma o rozmiarze  $m$  wykorzystujący  $k$  funkcji haszujących dla zbioru  $n$ -elementowego. Wówczas prawdopodobieństwo popełnienia błędu (przy założeniu niezależności odpowiednich zmiennych losowych) wynosi

$$\left(1 - \left(1 - \frac{1}{m}\right)^{kn}\right)^k \approx \left(1 - e^{-kn/m}\right)^k.$$



## Błędy i filtry Blooma

Korzystanie z filtrów Blooma naraża nas tylko na błędy I rodzaju, czyli wyniki fałszywie dodatnie.

Oznacza to, że sprawdzając, czy zadana wartość należy do zbioru, możemy otrzymać odpowiedź „tak” nawet wtedy, gdy nie jest to prawdą. Nigdy natomiast nie uzyskamy odpowiedzi „nie”, jeśli dana wartość w rzeczywistości do zbioru należy.

Rozważmy filtr Blooma o rozmiarze  $m$  wykorzystujący  $k$  funkcji haszujących dla zbioru  $n$ -elementowego. Wówczas prawdopodobieństwo popełnienia błędu (przy założeniu niezależności odpowiednich zmiennych losowych) wynosi

$$\left(1 - \left(1 - \frac{1}{m}\right)^{kn}\right)^k \approx \left(1 - e^{-kn/m}\right)^k.$$

Dla danych  $n$  i  $m$  optymalna liczba funkcja haszujących to  $\frac{m}{n} \ln 2$ .